RAG Framework Comparison — ISD Document Intelligence V5

Evaluated against deployment requirements: offline server, Ollama + gemma3:12b, MySQL, H100 GPU, Ubuntu 24.04

1. Overall Comparison

Factor	Custom RAG (Current)	LlamaIndex	LangChain
Approach	Hand-built pipeline using pypdf, ChromaDB, Ollama, MySQL	High-level RAG framework with built-in connectors	General-purpose LLM chaining framework with RAG support
Offline Operation	**Fully offline — no external calls after setup**	Mostly offline — some components may call home for telemetry	Mostly offline — some integrations require internet
Ollama Integration	**Direct API calls to localhost — full control**	**Native Ollama connector available**	**Native Ollama connector available**
PDF Extraction	pypdf (digital PDFs) / pdfplumber (complex layouts)	**Built-in readers: pypdf, pdfplumber, Docling, Unstructured**	**Document loaders: pypdf, Unstructured, Docling**
ChromaDB Support	**Direct ChromaDB API — full control over collections**	**Native ChromaDB vector store integration**	**Native ChromaDB vector store integration**
MySQL / Structured Search	**Full SQL — NL-to-SQL, aggregate queries, structured filters**	Requires custom SQL tool — not native	Requires SQLDatabaseChain — some setup needed
Hybrid Search (Vector + BM25 + SQL)	**Custom RRF fusion — fully tuned for this app**	Built-in hybrid search — less customizable	Ensemble retriever — configurable but complex
Chunking Control	**Full control — chunk size, overlap, custom logic**	Configurable but abstracted	Configurable but abstracted
Custom Prompting	**Direct prompt engineering — full control**	Template system — needs learning	PromptTemplate — flexible but verbose
Package Size / Dependencies	**Lean — ~15 core packages**	Heavy — 100+ transitive dependencies	Very heavy — 150+ transitive dependencies
Install Size	**~200 MB**	~800 MB	~1.2 GB

Factor	Custom RAG (Current)	LlamaIndex	LangChain
Version Stability	**Stable — no framework updates to track**	Frequent breaking changes between versions	Very frequent breaking changes — known issue
Entity / Timeline / Location Extraction	**Custom modules (entity_graph.py, activity_timeline.py, location_extractor.py)**	Possible but requires custom tool nodes	Possible with custom chains — more complex
Answer Rating System	**Built-in — 5-point scale, stores as training data**	Not built-in — custom development needed	Not built-in — custom development needed
Case-Scoped Search	**Native — per-case ChromaDB collections with fallback**	Possible with metadata filtering	Possible with metadata filtering
Observability / Debugging	**Direct — log exactly what you want**	Callbacks available but abstracted	LangSmith (requires internet) or custom callbacks
Learning Curve	**Low — standard Python + FastAPI**	Medium — framework-specific concepts	High — complex abstractions, frequent API changes
Production Readiness (offline)	**Proven — currently running on H100 server**	Viable — needs validation	Viable — needs validation
Multi-LLM Support	**Configurable via .env (gemma3:12b, llama3.3:70b, qwen2.5:72b)**	**Native multi-LLM support**	**Native multi-LLM support**
Community / Support	Self-maintained	**Large community, good docs**	**Very large community, extensive docs**

2. PDF Extraction Comparison

Factor	pypdf (Current)	pdfplumber	Docling
Digital PDFs	**Excellent**	**Excellent**	**Excellent**
Scanned PDFs (OCR)	Not supported	Not supported	**Supported (ML-based OCR)**
Complex Tables	Basic	**Good table extraction**	**Excellent**

Factor	pypdf (Current)	pdfplumber	Docling
Multi-column Layouts	Sometimes incorrect order	**Better reading order**	**Excellent**
Offline Operation	**Fully offline**	**Fully offline**	Downloads models from HuggingFace on first run
Install Size	**Small (~2 MB)**	**Small (~5 MB)**	Very large (~2 GB with models)
Processing Speed	**Fast**	**Fast**	Slower (ML inference per page)
Suitable for SMAC/IR Files	**Yes — digital government PDFs**	**Yes — better for tables**	Overkill — internet required for setup

3. Recommendation Summary

Decision	Recommendation
PDF Extractor	**pypdf** — sufficient for digital PDFs. Upgrade to **pdfplumber** if table extraction quality is poor during UAT.
RAG Framework	**Custom RAG (current)** — lean, fully offline, proven in production. Migrate to LlamaIndex only after go-live if a specific gap is identified.
LangChain	Not recommended — too heavy, frequent breaking changes, no clear benefit over current setup.
Docling	Not recommended — requires internet for model download. Not viable for offline server deployment.
Post Go-Live PoC	Evaluate **LlamaIndex + pdfplumber** as a potential V6 upgrade if document variety increases (scanned PDFs, complex tables).

Current Stack Decision: pypdf + Custom RAG pipeline is the right choice for V5 production deployment on the offline H100 server. It is lean, fully offline, proven, and meets all functional requirements. The custom pipeline gives full control over hybrid search, case-scoped collections, entity extraction, and the answer rating system — features that would require significant custom development in LlamaIndex or LangChain anyway.

Generated for: ISD Document Intelligence V5 | Server: Ubuntu 24.04, H100 GPU, 1TB RAM | LLM: gemma3:12b / llama3.3:70b via Ollama | Date: March 2026